56
Target all elements by tag name

Suppose you want to give your users the option of doubling the size of the text in all the paragraphs of the document. To accomplish this, you could give each paragraph an id. When the user clicks the **Double Text Size** button, a function would cycle through all the paragraph ids and make the change. The function could upsize the font either by assigning a new class to each paragraph, with the class specifying a larger font size, or it could assign the new style property directly, like this:

```
getElementById(the id).style.fontSize = "2em";
```

An easier way to accomplish the same thing is to have the function say, "Find all the paragraphs in the document and increase their size."

This statement finds all the paragraphs and assigns them to a variable:

```
var par = document.getElementsByTagName("p");
```

The variable, `par`, now holds an array-like collection of all the paragraphs in the document. Suppose, to keep things simple, that the body of the document comprises just three paragraphs.

```
<p>This bed is too small.</p>
<p>This bed is too big.</p>
<p>This bed is just right.</p>
```

`par[0]` is the first paragraph. `par[1]` is the second paragraph. `par[2]` is the third paragraph. `par.length` is 3, the number of items in the collection.

If you write...

```
var textInMiddleParagraph = par[1].innerHTML;
```

...the variable `textInMiddleParagraph` is assigned "This bed is too big."

If you write...

```
par[1].innerHTML = "This SUV is too big.";
```

...the paragraph text changes to "This SUV is too big."

Here's a loop that assigns a font family to all the paragraphs.

```
1 for (var i = 0; i < par.length; i++) {
2   par[i].style.fontFamily = "Verdana, Geneva, sans-serif";
3 }
```

This statement makes a collection of all the images in the document and assigns it to the variable `pics`.

```
var pics = document.getElementsByTagName("img");
```

This statement makes a collection of all the divs in the document and assigns it to the variable `divs`.

```
var divs = document.getElementsByTagName("div");
```

This statement makes a collection of all the unordered lists in the document and assigns it to the variable `ulists`.

```
var ulists = document.getElementsByTagName("ul");
```

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/56.html

57
Target some elements by tag name

In the last chapter you learned how to make an array-like collection of all the elements in a document that have a particular tag name. For example, this statement makes a collection of all the paragraphs in the document and assigns the collection to the variable `pars`.

```
var pars = document.getElementsByTagName("p");
```

You also learned to read or set the properties of any of the elements in a collection of elements by using array-like notation—a number in square brackets. For example, this statement "reads" the contents of the second element in the collection `pars` and assigns the string to the variable `textInMiddleParagraph`.

```
var textInMiddleParagraph = pars[1].innerHTML;
```

But suppose you don't want a collection of all the paragraphs in the document. Suppose you want just a collection of all the paragraphs within a particular div. No problem. Let's say the id of the div is "rules". Here's the code.

```
1 var e = document.getElementById("rules");
2 var paragraphs = e.getElementsByTagName("p");
```

Line 1 assigns the div with the id "rules" to the variable `e`.

Line 2 makes a collection of all the paragraphs in the div and assigns the collection to the variable `paragraphs`.

Once the id of the target div has been assigned to a variable, for example `e`, you can assemble a collection of all the paragraphs within that div. Instead of writing...

```
document.getElementsByTagName("p");
```

...which would make a collection of all the paragraphs in the document, you write...

```
e.getElementsByTagName("p");
```

...which makes a collection of all the paragraphs in the div.

Suppose you have a table with a white background. When the user clicks a button, the cells turn pink. Here's the code.

```
1 var t = document.getElementById("table9");
2 var cells = t.getElementsByTagName("td");
3 for (var i = 0; i < cells.length; i++) {
4   cells[i].style.backgroundColor = "pink";
5 }
```

Here's the line-by-line breakdown.

1. Targets the table by its id
2. Assembles a collection of all the elements in the table with a `td` tag

3-5. Loops through all of them to change their background color

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/57.html

58
The DOM

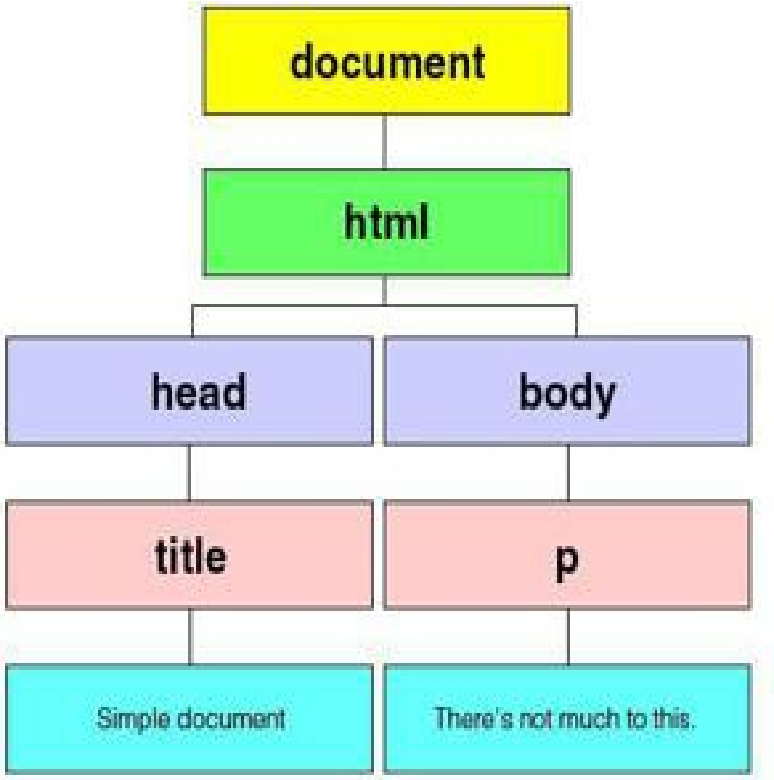
In previous chapters you learned two different ways to target components of your web page so you could read or set them. You learned to `getElementById`, and you learned to `getElementsByTagName`. These are often the best methods for targeting things, but they have limitations. The first, `getElementById`, gives you access only to those components that have been assigned an id. The second, `getElementsByTagName`, is good for wholesale changes, but is a bit cumbersome for fine surgical work. Both approaches can change things on your web page, but neither is able to deal with *all* the things on the page, to create new things, to move existing things, or to delete them.

Fortunately, both of these approaches are only two of many methods for working with the **Document Object Model**, the **DOM**. The DOM is an organization chart, created automatically by the browser when your web page loads, for the whole web page. All the things on your web page—the tags, the text blocks, the images, the links, the tables, the style attributes, and more—have spots on this organization chart. This means that your JavaScript code can get its hands on anything on your web page, anything at all, just by saying where that thing is on the chart. What's more, your JavaScript can add things, move things, or delete things by manipulating the chart. If you wanted to (you wouldn't), you could almost create an entire web page from scratch using JavaScript's DOM methods.

Here's a simplified web page. I've indented the different levels in the hierarchy. The three top levels of the DOM are always the same for a standard web page. The document is the first level. Under the document is the second level, the html. And under the html are the co-equal third levels, the head and the body. Under each of these are more levels.

```
1st level: document
2nd level:   <html>
3rd level:     <head>
4th level:       <title>
5th level:         Simple document
                 </title>
               </head>
3rd level      <body>
4th level        <p>
5th level          There's not much to this.
                 </p>
               </body>
             </html>
```

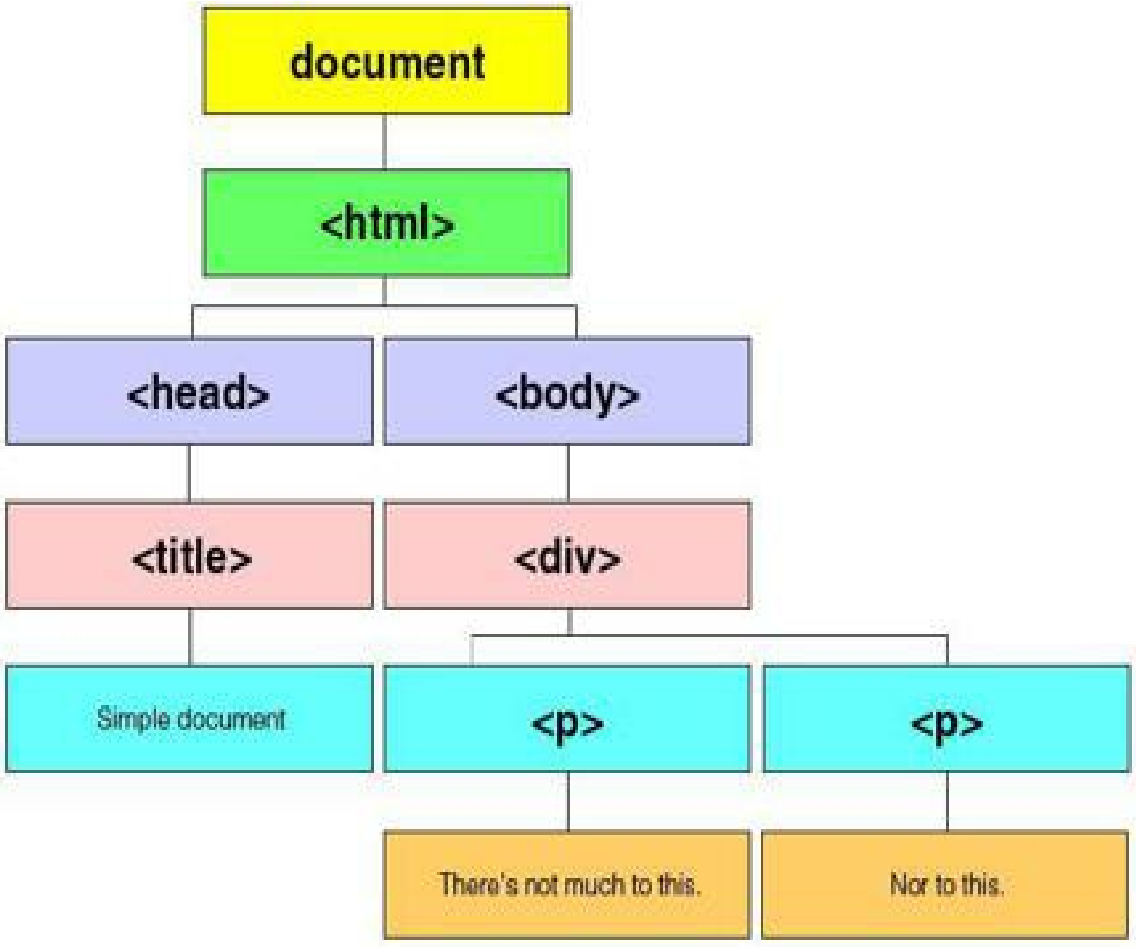
Here's the same thing, shown as an organization chart. Note that this is an idealized chart, cleaned of junk artifacts that most browsers insert in the DOM. I'll show you how to clean out these artifacts in later chapters.



As you can see, every single thing on the web page is included, even the title text and the paragraph text. Let's make it a little more complicated by adding a div and a second paragraph. Here it is in HTML form.

```
1st level: document
2nd level:   <html>
3rd level:     <head>
4th level:       <title>
5th level:         Simple document
                 </title>
               </head>
3rd level      <body>
4th level        <div>
5th level          <p>
6th level            There's not much to this.
                   </p>
5th level          <p>
6th level            Nor to this.
                   </p>
                 </div>
               </body>
             </html>
```

And in organization chart (minus any junk artifacts)...



In a company organization chart, each box represents a person. In the DOM organization chart, each box represents a **node**. The HTML page represented above, in its cleaned-up DOM form, has 11 nodes: the document node, the html node, the head and body nodes, the title node, the div node, two paragraph nodes, and three text nodes, one for the title and one for each of the two paragraphs.

In this particular chart, there are three types of nodes: document, element, and text. The document node is the top level. Element nodes are **<html>**, **<head>**, **<body>**, **<title>**, ** **, and **<p>**. Text nodes are the strings that comprise the title and the two paragraphs.

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/58.html

59
The DOM:
Parents and children

Can you name the second child of the 44th President of the United States? That would be Sasha. How about the parent (male) of the 43rd President? That's right, George.

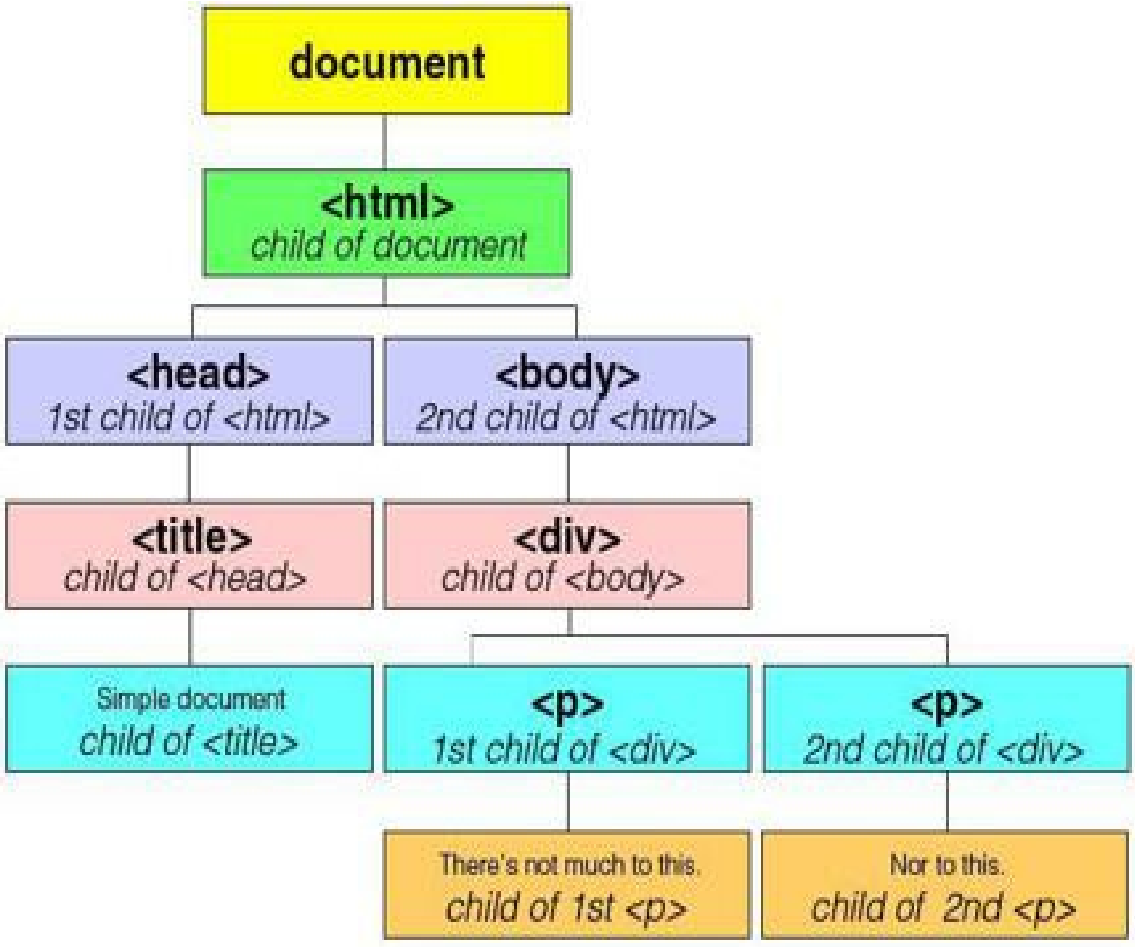
Welcome to the most fundamental way of designating nodes of the Document Object Model (DOM). You can designate any node of the DOM by saying the node is the *x*th child of a particular parent. You can also designate a node by saying it's the parent of any child.

Take a look at the simplified html document from the last chapter.

```
1st level: document
2nd level:   <html>
3rd level:     <head>
4th level:       <title>
5th level:         Simple document
                 </title>
               </head>
3rd level      <body>
4th level        <div>
5th level          <p>
6th level            There's not much to this.
                   </p>
5th level          <p>
6th level            Nor to this.
                   </p>
                 </div>
               </body>
             </html>
```

Except for the document node, each node is enclosed within another node. The **<head>** and **<body>** nodes are enclosed within the **<html>** node. The ** ** node is enclosed within the **<body>** node. Two **<p>** nodes are enclosed within the ** ** node. And a text node is enclosed within each of the **<p>** nodes.

When a node is enclosed within another node, we say that the enclosed node is a **child** of the node that encloses it. So, for example, the ** ** node is a child of the **<body>** node. Conversely, the **<body>** node is the **parent** of the ** ** node. Here's the organization chart from the last chapter, again cleaned of junk artifacts, showing all the parents and their children.



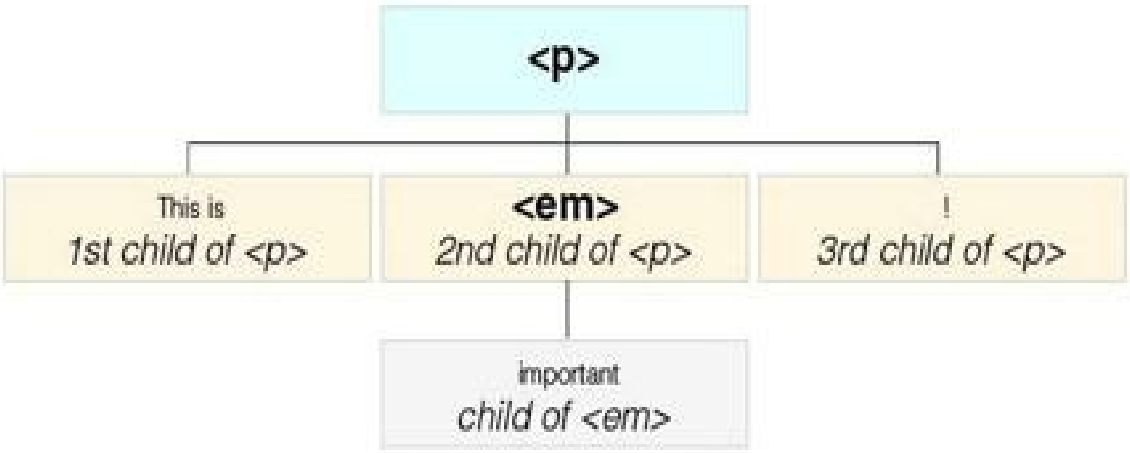
As you can see, **<html>** is the child of the **document**...**<head>** and **<body>** are the children of **<html>**...** ** is the child of **<body>**...two **<p>**'s are the children of ** **...each text node is the child of a **<p>**. Conversely, the **document** is the parent of **<html>**, **<html>** is the parent of **<head>** and **<body>**, **<head>** is the parent of **<title>**, **<title>** is the parent of a text node, and so on. Nodes with the same parent are known as siblings. So, **<head>** and **<body>** are siblings because **< html>** is the parent of both. The two **<p>**'s are siblings because ** ** is the parent of both.

Starting at the bottom of the chart, the text "Nor to this." is a child of **<p>**, which is a child of ** **, which is a child of **<body>**, which is a child of **<html>**, which is a child of the **document**.

Now look at this markup.

```
<p>This is <em>important</em>!</p>
```

If you made a chart for this paragraph, you might think that all the text is a child of the **<p>** node. But remember, every node that is enclosed by another node is the child of the node that encloses it. Since the text node "important" is enclosed by the element node ****, this particular text node is the child of ****, not **<p>**. The text nodes "This is " and "!" as well as the element node **** are siblings, because they're all enclosed by **<p>**. They're all children of **<p>**.


<p>
This is
1st child of <p>

2nd child of <p>
!
3rd child of <p>
important
child of

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/59.html

60
The DOM:
Finding children

As you learned in the last chapter, the Document Object Model (DOM) is a hierarchy of parents and children. Everything on the web page, excluding the document itself, is a child of something else. So now let's talk about how your JavaScript code can make use of this hierarchy of parents and children to read or change virtually anything on a web page.

Let's begin with the methods you've already learned for targeting things on the web page. You'll recall from earlier chapters that you can target an element by specifying its id, if it has one.

```
var eField = document.getElementById("email");
```

The statement above targets the element with the id of "email".

You also learned how to make an array-like collection of all the elements of a particular kind within the document...

```
var eField = document.getElementsByTagName("p");
```

Having made a collection of paragraphs, you can target any paragraph within the collection so that you can, for example, read its contents.

```
var contents = p[2].innerHTML;
```

The statement above assigns the text string contained within the third paragraph of the document to the variable `contents`.

An alternative to listing all the elements of a certain kind in the document is to narrow the focus below the document level, for example to a div, and then make a collection within that div.

```
1 var d = document.getElementById("div3");
2 var p = d.getElementsByTagName("p");
3 var contents = p[2].innerHTML;
```

In the example above, you a generate a collection not of all the paragraph elements in the document, but only those paragraph elements within the div that has an id of "div3". Then you target one of those paragraphs.

Now consider this markup.

```
<body>
  <div id="cal">
     <p>Southern Cal is sunny.</p>
```

```
    <p>Northern Cal is rainy.</p>
    <p>Eastern Cal is desert.</p>
  </div>
  <div id="ny">
    <p>Urban NY is crowded.</p>
    <p>Rural NY is sparse.</p>
  </div>
</body>
```

If you wanted to read the contents of the last paragraph in the markup, you could write...

```
1 var p = document.getElementsByTagName("p");
2 var contents = p[4].innerHTML;
```

Line 1 makes a collection of all the `<p>`s in the document and assigns the collection to the variable `p`. Line 2 "reads" the text contained in the 5th paragraph of the document.

Another approach: You could target the same paragraph by narrowing the collection of elements to those that are just in the second div.

```
1 var div = document.getElementById("ny");
2 var p = div.getElementsByTagName("p");
3 var contents = p[1].innerHTML;
```

Here's the breakdown:

1. Assigns the div with the id "ny" to the variable `div`
2. Makes a collection of all the `<p>`s in the div and assigns the collection to the variable `p`
3. "Reads" the text contained in the 2nd paragraph of the div and assigns it to the variable `contents`

Now I'll show you a new way to target the paragraph, by using the DOM organization chart.

```
1 var p = document.childNodes[0].childNodes[1].childNodes[1].childNodes[1];
2 var contents = p.innerHTML;
```

I'll highlight each child level and tell you what it points to.

1. `document.childNodes[0].childNodes[1].childNodes[1].childNodes[1];` is the first child of the document, `<html>`
2. `document.childNodes[0].childNodes[1].childNodes[1].childNodes[1];` is the second child of `<html>`, `<body>`
3. `document.childNodes[0].childNodes[1].childNodes[1].childNodes[1];` is the second child of `<body>`, `<div>` with the id "ny"
4. `document.childNodes[0].childNodes[1].childNodes[1].childNodes[1];` is the second child of `<div>` with the id "ny", the second `<p>` within the div

Note: The code above assumes the browser hasn't included any junk artifacts in the DOM. I'll discuss these in the next chapter.

As we work our way down the organization chart, each parent is followed by a dot, which is followed by the keyword `childNodes`, which is followed by a number in brackets, as in array notation. As in array notation, the first child is number 0.

Here's the markup again. The key players in the code above are highlighted.

```
<body>
  <div id="cal">
    <p>Southern Cal is sunny.</p>
    <p>Northern Cal is rainy.</p>
    <p>Eastern Cal is desert.</p>
   </div>
   <div id="ny">
     <p>Urban NY is crowded.</p>
     <p>Rural NY is sparse.</p>
   </div>
</body>
```

In the example above, I showed you how to work your way down to the targeted node starting at the document level. But in practice, you'd normally start at a lower level. For example, you could start with the second div, specifying its id. Then you'd target one of its children.

```
1 var d = document.getElementById("ny");
2 var p = d.childNodes[1];
3 var contents = p.innerHTML;
```

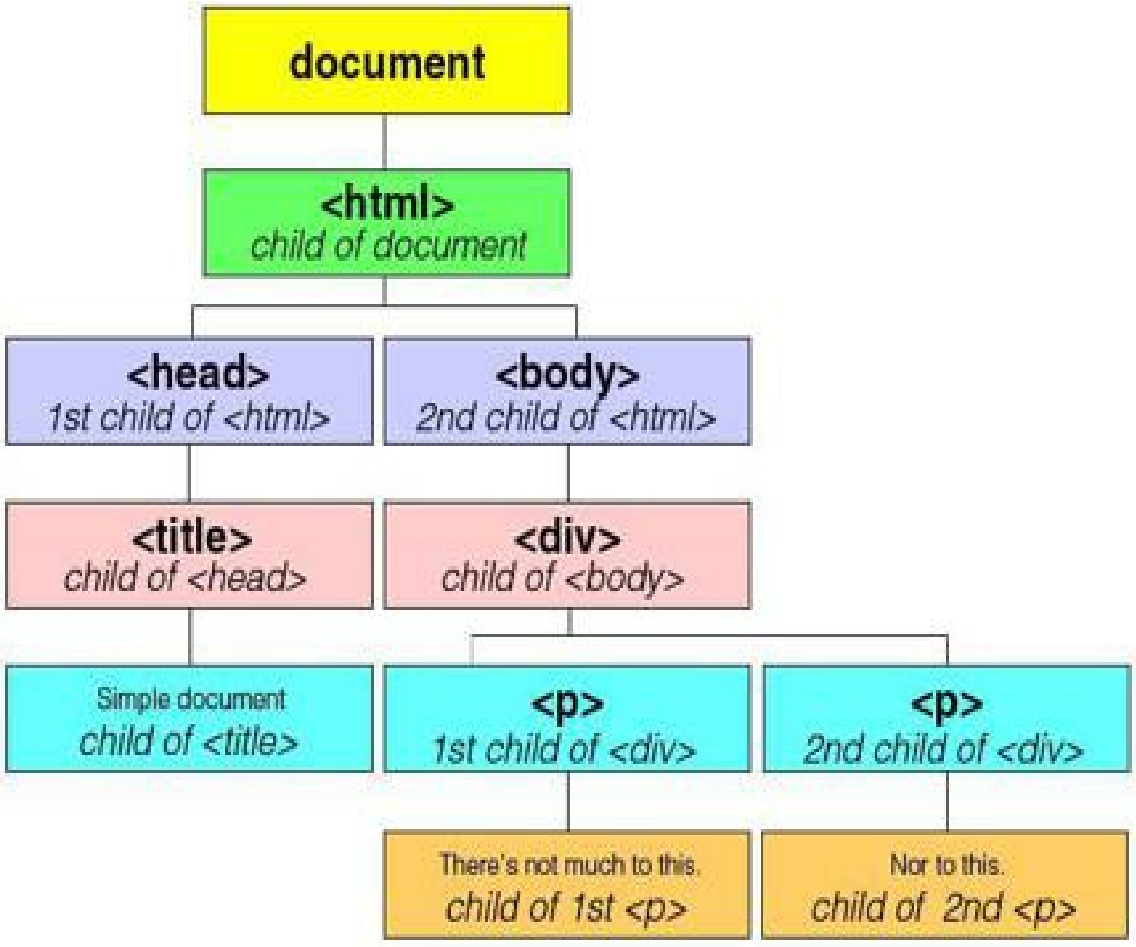
Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/60.html

61
The DOM:
Junk artifacts and nodeType

It's time to talk about the junk artifacts of the DOM, the pesky things that I've been referring to in the last three chapters. Let's revisit the simplified web page that we've been using as an example.

```
1st level: document
2nd level:   <html>
3rd level:     <head>
4th level:       <title>
5th level:         Simple document
                 </title>
               </head>
3rd level      <body>
4th level        <div>
5th level          <p>
6th level            There's not much to this.
                   </p>
5th level          <p>
6th level            Nor to this.
                   </p>
                 </div>
               </body>
             </html>
```

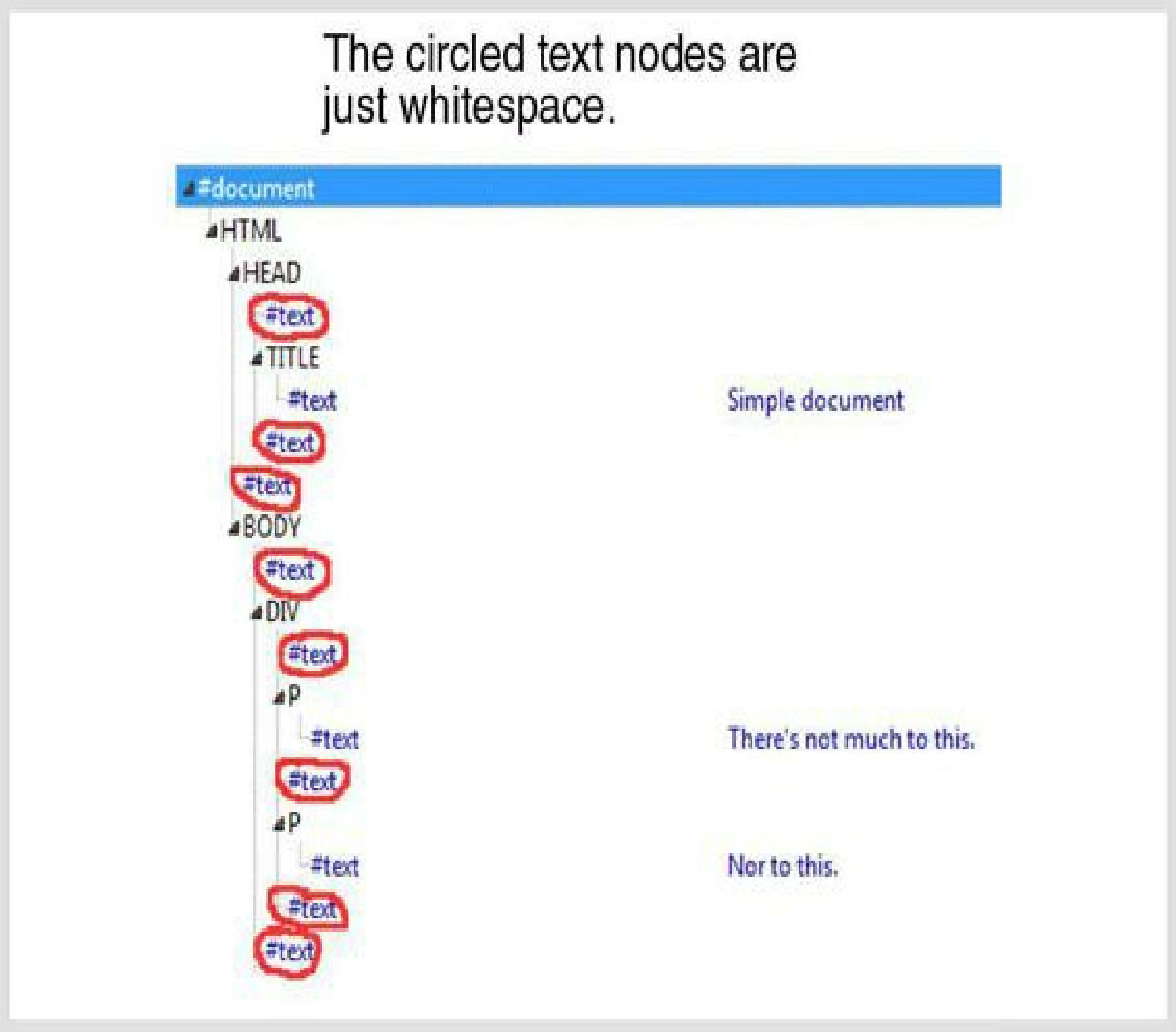
This is how I diagrammed the DOM for the page.



But this diagram is valid for only a couple of browsers. Most browsers interpret the whitespace that's created by some carriage returns, tabs, and spaces *used to format the code* as text nodes. When *you* look at the markup, you see 3 text nodes, the children of the title element and of the two paragraph elements.

```
1st level: document
2nd level:   <html>
3rd level:     <head>
4th level:       <title>
5th level:         Simple document
                 </title>
               </head>
3rd level      <body>
4th level        <div>
5th level          <p>
6th level            There's not much to this.
                   </p>
5th level          <p>
6th level            Nor to this.
                   </p>
                 </div>
               </body>
             </html>
```

Firefox sees this markup with 8 additional text nodes that are nothing but whitespace created by indenting.



These extra text nodes create noise that makes it hard for your DOM-reading code to find the signal. In one browser, the first child of the body node might be a div. In another browser, the first child of the body might be an empty text node.

There are a number of solutions. As one approach, the Mozilla Developer Network suggests a workaround that's almost comically desperate but does solve the problem without any extra effort from JavaScript. You format the markup like this.

```
<html
  ><head
    ><title
      >Simple document
    </title
  ></head
  ><body
    ><div
      ><p
```

```
        >There's not much to this.
      </p
      ><p
       >Nor to this.
      </p
    ></div
  ></body
></html>
```

This markup takes advantage of the fact that any carriage returns, tabs, and spaces that are enclosed in the < and the > of a tag are ignored by the browser. To the browser, < p > is the same as <p>. And...

```
<
div
>
```

...is the same as .

When whitespace is inside the brackets, the browser doesn't count it as a text node. So if you "hide" all carriage returns, tabs, and spaces inside the brackets as the code above does, there's no junk in the DOM. Any noise that would keep your JavaScript from reading the signal is removed.

You can also clean out junk in the DOM by using a minifier program like the one at http://www.willpeavy.com/minifier/. You will, of course, want to preserve the original, non-minified version of the file so you can revise the page in the future without going crazy.

```
<html><head><title>Simple document</title></head><body><div><p>There's not much to this.</p><p>Nor to this.
</p></div></body></html>
```

Another approach is to format your HTML conventionally and let your JavaScript sniff out the junk nodes. JavaScript can check a node to see what type it is—element, text, comment, and so on. For example, this statement checks the node type of a targeted node and assigns it to the variable nType.

```
var nType = targetNode.nodeType;
```

In the statement above, JavaScript assigns a number representing the node type to the variable nType. If the node is an element like or <p>, the number is 1. If it's a text node, the number is 3.

Suppose you want to replace the text content of the second paragraph in a particular div with the string "All his men." Here's the markup, with the text we're going to change highlighted.

```
<div id="humpty">
  <p>All the king's horses.</p>
  <p>All the dude's crew.</p>
  <p>All the town's orthopedists.</p>
</div>
```

This is the code.

```
1  var d = document.getElementById("humpty");
```

```
2  var pCounter = 0;
3  for (var i = 0; i < d.childNodes.length; i++) {
4    if (d.childNodes[i].nodeType === 1 ) {
5      pCounter++;
6    }
7    if (pCounter === 2) {
8      d.childNodes[i].innerHTML = "All his men.";
9      break;
10   }
11 }
```

Here's the breakdown:

1 Assigns the div to the variable `d`

2 Counts the number of paragraphs

3 Goes through the children of the div looking for the next element node, i.e. Type 1, which we assume is a paragraph

4-6 Adds 1 to the counter. We're looking for the second paragraph.

7-9 When the counter hits 2, we've reached the targeted paragraph. Change the text

Of course, if you know you might want your JavaScript to change that second paragraph, it would be a lot easier to assign it an id, and go straight to it, like this.

```
document.getElementById("p2").innerHTML = "All his men.";
```

You can see why the `getElementId` method is more popular with coders than tracing the DOM hierarchy. But there are some things you can't do without working your way through the parent-child relationships of the DOM, as you'll learn in subsequent chapters.

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/61.html

62
The DOM:
More ways to target elements

So far you've learned to use the DOM's parent-child hierarchy to target a child node of a parent node by specifying its order in an array-like collection of children—`childNodes[0]`, `childNodes[1]`, `childNodes[2]`, and so on. But there are other ways to use the hierarchy for targeting. To begin with, instead of writing `childNodes[0]`, you can write `firstChild`. And instead of writing, for example, `childNodes[9]`, to target the 10th child in a 10-child collection of children, you can write `lastChild`.

```
var targetNode = parentNode.childNodes[0];
```

...is the same as...

```
var targetNode = parentNode.firstChild;
```

And if there are, for example, 3 child nodes...

```
var targetNode = parentNode.childNodes[2];
```

...is the same as...

```
var targetNode = parentNode.lastChild;
```

You can also work backwards, targeting the parent node of any child node. Let's say you have this markup. Using the DOM, how do you find the parent of the highlighted div?

```
<div id="div1">
  <div id="div2">
    <p>Chicago</p>
    <p>Kansas City</p>
    <p>St. Louis</p>
  </div>
</div>
```

You want to know the parent of the div with the id "div2". The following code assigns the parent of the "div2" div to the variable `pNode`.

```
1 var kidNode = document.getElementById("div2");
2 var pNode = kidNode.parentNode;
```

You can use `nextSibling` and `previousSibling` to target the next child and the previous child in the collection of an element's children. In the following code, the first statement targets a div with the id "div1". The second statement targets the next node that has

the same parent as the "div1" id.

```
1 var firstEl = document.getElementById("div1");
2 secondEl = firstEl.nextSibling;
```

If there is no `nextSibling` or `previousSibling`, you get null. In the following code, the variable `nonexistentEl` has a value of null, because JavaScript finds that there is no previous node that has the same parent as `firstEl`.

```
1 var firstEl = document.getElementById("div1");
2 var nonexistentEl = firstEl.previousSibling;
```

Counting siblings can be tricky, because, as you know, some browsers treat whitespace created by HTML formatting as text nodes, even though they're empty and without significance.

In the HTML example above, with two divs and three paragraphs, you might think that the next sibling of the first paragraph is the second paragraph. But actually, in some browsers, the first paragraph's next sibling is an empty text node. The next sibling of *that* node is the second paragraph.

What this means is that in order to target a node with `nextSibling` or `previousSibling`, you have to either format your HTML markup defensively as I showed you in the last chapter, or else type-test each node to make sure it's the kind you're looking for, as I also showed you in the last chapter.

Once again, I'll say that you're often better off assigning an id to any node you might want to "read" or change. Then you can target the node more directly, using `document.getElementById`.

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/62.html

63
The DOM:
Getting a target's name

In a previous chapter you learned how to get a node's node type with a statement like this.

```
var nType = targetNode.nodeType;
```

In the example above, if the node is an element, the variable `nType` will be assigned the number 1. If the node is text, it'll be assigned the number 3.

You can get additional information about a node by using `nodeName`. In the following example, the node name of the target node is assigned to the variable `nName`.

```
1 var parent = document.getElementById("div1");
2 var target = parent.firstChild;
3 nName = target.nodeName;
```

Lines 1 and 2 target the first child of an element with the id "div1". Line 3 assigns the node name of the target to the variable `nName`.

In the example above, if the node is an element, the variable `nName` will be assigned a string, like P or DIV, that corresponds to the characters of the tag that are enclosed in the brackets. In HTML, the name is usually given in all-caps, even if the markup is in lowercase.

Tag	Node Name
<p> *or* <P>	P
or <DIV>	DIV
or <SPAN>	SPAN
<img> *or* <IMG>	IMG
<a> *or* <A>	A
or <EM>	EM
<table> *or* <TABLE>	TABLE
or <LI>	LI

On the other hand, if the node is a text node, the name of the node is always #text—in lower-case.

If it's a text node, you can find out its value—i.e. its content—this way.

```
1 var parent = document.getElementById("div1");
2 var target = parent.firstChild;
```

```
3 var nTextContent = target.nodeValue;
```

Lines 1 and 2 target the first child of an element with the id "div1". Line 3 assigns the node value to the variable `nTextContent`.

So if this is the markup, with the first child of the H2 element a node with the name "#text"...

```
<h2>Do <em>not</em> hit!</h2>
```

...the node value is "Do ".

An element node like P or IMG has a name but no value. If you try to assign an element node value to a variable, the variable will be assigned null.

```
<h2>Do <em>not</em> hit!</h2>
```

In the above example, `<em>` is an element, which means its value is null.

It's possible to confuse the node value of a text node with the `innerHTML` property. There are two differences.

- `innerHTML` is a property of the element node, <h2>, in the above example. The node value is a property of the text node itself, not the parent element.
- `innerHTML` includes *all* the descendants of the element, including any inner element nodes like as well as text nodes. The node value includes only the characters that comprise the single text node.

In the following code, the `innerHTML` of the H2 element is highlighted.

```
<h2>Do <em>not</em> hit!</h2>
```

In the following code, the node value of the first child of the H2 element is highlighted.

```
<h2>Do <em>not</em> hit!</h2>
```

Since there are situations in which `nodeName` gets you, for example, a "p" instead of a "P" or an "href" instead of an "HREF," it's good practice to standardize the case when you're testing for names, like this.

```
if (targetNode.nodeName.toLowerCase === "img") {
```

If the node name was originally upper-case, the code changes it to lower-case. If it was lower-case to begin with, no harm done.

If you're checking a node name to see if a node is a text node, converting to lower-case isn't necessary, because the name of a text node is always "#text", never "#TEXT".

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/63.html

64
The DOM:
Counting elements

In an earlier chapter you learned how to make an array-like collection of all the elements that share a particular tag name. For example, the following code makes a collection of all the `<li>` elements and assigns the collection to the variable `liElements`.

```
var liElements = getElementsByTagName("li");
```

Once you have the collection of elements, you can find out how many of them there are. In the following code, the number of `<li>` elements in the collection represented by `liElements` is assigned to the variable `howManyLi`.

```
var howManyLi = liElements.length;
```

Then you can, for example, cycle through the collection looking for `<li>` elements that have no text, and can enter some placeholder text.

```
1 for (var i = 0; i < howManyLi; i++) {
2   if (liElements[i].innerHTML === "") {
3     liElements[i].innerHTML = "coming soon";
4   }
5 }
```

I've walked you through this to introduce you to an analagous move you can make using the DOM hierarchy rather than tag names. You can make a collection of all the child nodes of a targeted node.

```
1 var parentNode = document.getElementById("d1");
2 var nodeList = parentNode.childNodes;
```

Line 1 assigns an element with the id "d1" to the variable `parentNode`. Line 2 assigns a collection of the child nodes of `parentNode` to `nodeList`.

You can get the number of items in the collection. The following statement assigns the number of items in the node collection to the variable `howManyKids`.

```
var howManyKids = nodeList.length;
```

Then you can target any item in the collection. The following code counts the number of images within the div.

```
1 var numberPics = 0;
2 for (var i = 0; i < howManyKids; i++) {
```

```
3   if (nodelist[i].nodeName.toLowerCase() === "img") {
4     numberPics++;
5   }
6 }
```

Here's the breakdown:

1 Declares the image-counter and sets it at 0

2 Loops through all the children of the div

3-5 If the node name, converted to lower-case, is "img", increments the image-counter

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/64.html

65
The DOM: Attributes

Take a look at this bit of markup.

```
<a href="http://www.amazon.com">Shop</a>
```

You've learned that in the above markup the text node "Shop" is the first (and only) child of the element node `<a>`. But what is `href="http://www.amazon.com"`? It's definitely a node of the DOM, and you could say that it's subsidiary to `<a>`. But it's not a child of `<a>`. It's an *attribute* of `<a>`.

Whenever you see this form...

```
<element something="something in quotes">
```

...you're looking at an element with an attribute. The equal sign and the quotes are the tipoff. The word on the left side of the equal sign is the attribute *name*. The word on the right side of the equal sign is the attribute *value*.

Here are more examples. The attribute values are highlighted.

```
<div id="p1">
<p class="special">
<img src="images/slow-loris.png">
<span style="font-size:24px;">
```

An element can have any number of attributes. In this tag, the element `img` has 4 attributes. I've highlighted their values.

```
<img src="dan.gif" alt="Dan" height="42" width="42">
```

You can find out whether an element has a particular attribute with `hasAttribute`.

```
1 var target = document.getElementById("p1");
2 var hasClass = target.hasAttribute("class");
```

Line 1 assigns the element to a variable, `target`. Line 2 checks to see if the element has an attribute called "class". If it does, the variable `hasClass` is assigned true. If not, it is assigned false.

You can read the value of an attribute with `getAttribute`.

```
1 var target = document.getElementById("div1");
2 var attVal = target.getAttribute("class");
```

Line 1 assigns the element to a variable, `target`. Line 2 reads the value of the attribute and assigns it to the variable `attVal`.

You can set the value of an attribute with `setAttribute`.

```
1 var target = document.getElementById("div1");
2 target.setAttribute("class, "special");
```

Line 1 assigns the element to a variable, `target`. Line 2 gives it an attribute "class" (the first specification inside the parens) with a value of "special" (the second specification inside the parens). In effect, the markup becomes:

```
<div id="div1" class="special">
```

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/65.html

66
The DOM:
Attribute names and values

In previous chapters you learned how to make a collection of elements that share the same tag name, with a statement like this...

```
var list = document.getElementsByTagName("p");
```

...and how to make a collection of all the children of an element, with a statement like this...

```
var list = document.getElementById("p1").childNodes;
```

Similarly, you can make a collection of all the attributes of an element, with a statement like this...

```
var list = document.getElementById("p1").attributes;
```

With the collection assigned to a variable, you can get the number of items in the collection...

```
var numOfItems = list.length;
```

Alternatively, you could compress these tasks into a single statement, without assigning the target to a variable first.

```
var numOfItems = document.getItemById("p1").attributes.length;
```

Using array-like notation, you can find out the name of any attribute in the collection. The following statement targets the third item in the collection of attributes and assigns its name to the variable `nName`.

```
var nName = list[2].nodeName;
```

For example, if the markup is...

```
<p id="p1" class="c1" onMouseover="chgColor();">
```

...the variable `nName` is assigned "onMouseover".

You can also get the value of the attribute...

```
var nValue = list[2].nodeValue;
```

In the example markup above, the variable `nValue` is assigned "chgColor();".

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/66.html

67
The DOM: Adding nodes

Using the DOM hierarchy, you can add element, attribute, and text nodes anywhere in the head or body sections of a document. In this chapter you'll learn how to create a paragraph node, give it an attribute, and fill it with text content. In the next chapter, you'll learn how to insert the paragraph, along with its attributes and text content, into the page.

The first step is to create the paragraph node.

```
var nodeToAdd = document.createElement("p");
```

The example creates a paragraph element. To create a div element, you'd put "div" in the parentheses. To create an image element, you'd put "img" there. To create a link, you'd put "a" there. And so on.

Here's a statement that creates an image element.

```
var imgNodeToAdd = document.createElement("img");
```

In the last chapter, you learned how to add an attribute to an element that's already part of the web page, using `setAttribute`. You can do the same thing with an element that you've created but haven't yet placed in the document body.

```
nodeToAdd.setAttribute("class", "regular");
```

The code above gives the new paragraph element that you just created the class "regular". If you wanted to, you could add, using separate statements for each, more attributes to the paragraph element—for example, a span, a style, even an id.

If the element were an `<img>` element, you could add a border or an alt as an attribute. If it were an `<a>` element, you could add the web address.

This statement adds a border attribute to an image element.

```
imgNodeToAdd.setAttribute("border", "1");
```

The code above gives the image a 1-pixel border.

Remember, at this point, we're just creating nodes. The new attribute node has been connected to the new element node, but we haven't added the nodes to the page yet.

Getting back to the paragraph element, we've created it and added a class attribute to it. Now let's give it some text content. Again, we begin by creating the node we want.

```
var newTxt = document.createTextNode("Hello!");
```

The statement above creates a text node with the content "Hello!"

Next, we place the text into the paragraph element.

```
nodeToAdd.appendChild(newTxt);
```

The statement above adds the new text node, whose content is "Hello!", to the new paragraph node.

Now we have a new paragraph node with a class attribute and some text content. We're ready to insert it into the page.

Find the interactive coding exercises for this chapter at:
http://www.ASmarterWayToLearn.com/js/67.html

68
The DOM:
Inserting nodes

In the last chapter you learned how to add text content to an element by first creating a text node and then appending the node to the element with `appendChild`. You can of course use the same method to append the paragraph element itself, filled with content or not, to a parent node, for example the body or a div. This set of statements targets a div (line 1), creates a new paragraph element (line 2), creates the text to go in it (line 3), places the text into the paragraph (line 4) and then appends the paragraph element along with its text content to the targeted div (line 5).

```
1 var parentDiv = document.getElementById("div1");
2 var newParagraph = document.createElement("p");
3 var t = document.createTextNode("Hello world!");
4 newParagraph.appendChild(t);
5 parentDiv.appendChild(newParagraph);
```

When you add a node to your web page by appending it as a child to a parent, as I did in the example above, the limitation is that you have no control over where the new element lands among the parent's children. JavaScript always places it at the end. So for example, if the div in the example above already has three paragraphs, the new paragraph will become the fourth one, even if you'd like to place it in a higher position.

The solution is `insertBefore`.

For example, suppose you want the paragraph to be the first one in the div, rather than the last.

```
1 var parentDiv = document.getElementById("div1");
2 var newParagraph = document.createElement("p");
3 var t = document.createTextNode("Hello world!");
4 newParagraph.appendChild(t);
5 paragraph1 = parentDiv.firstChild;
6 parentDiv.insertBefore(newParagraph, paragraph1);
```

The example above selects one of the children of the parent element as the target (line 5), and tells JavaScript to insert the new element and its text content before that element (line 6).

There is no `insertAfter` method as such. But you can still do it. Simply `insertBefore` the target node's next sibling.

```
1 var target = parentDiv.childNodes[1];
2 parentDiv.insertBefore(newE, target.nextSibling);
```

By inserting the new node before the next sibling of the second child, you insert the new node after the second child.

To remove a node, use `removeChild`.

```
1 var parentDiv = document.getElementById("div1");
2 var nodeToRemove = parentDiv.childNodes[2];
3 parentDiv.removeChild(nodeToRemove);
```